Assignment 2 – Complete Solutions

Object Oriented Programming (C++) | LAB-03

Task 01 – Inventory Management System (15 Marks)

This solution uses an **InventoryItem** structure with an item code, item name, cost, and selling price. It stores 10 items, generates codes from the first three uppercase letters of each item name plus a running number, and calculates a 40% profit on cost. The final code uses a string item code as required by part 2 of the task.

Complete C++ code

```
#include <iostream>
#include <string>
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include <cctype>
using namespace std;

const int ITEM_COUNT = 10;

struct InventoryItem
{
    string itemCode;
    string itemName;
    double cost;
    double sellingPrice;
};

// Creates one inventory item and calculates its selling price.
InventoryItem insertItem()
{
    InventoryItem item;

    cout << "Enter item name: ";
    getline(cin >> ws, item.itemName);

    cout << "Enter item cost: ";
    cin >> item.cost;

    // Use the first three letters of the name in uppercase.
    string prefix = item.itemName.substr(0, 3);
    for (int i = 0; i < (int)prefix.length(); i++)
    {
        prefix[i] = (char)toupper((unsigned char)prefix[i]);
    }

    // Generate a running number and combine it with the prefix.
    static int runningNumber = 1;
    item.itemCode = prefix + "_" + to_string(3120 + runningNumber);
    runningNumber++;

    // Selling price includes 40% profit on the cost.
    item.sellingPrice = item.cost * 1.40;

    return item;
}

// Displays all inventory items in a readable table.
void displayStock(const InventoryItem* stock)
{
    cout << "\n----------- INVENTORY STOCK -----------\n";
    cout << left << setw(16) << "Item Code"
         << setw(22) << "Item Name"
         << right << setw(12) << "Cost"
         << setw(15) << "Selling Price" << endl;

    cout << fixed << setprecision(2);

    for (int i = 0; i < ITEM_COUNT; i++)
    {
        cout << left << setw(16) << stock[i].itemCode
             << setw(22) << stock[i].itemName
             << right << setw(12) << stock[i].cost
             << setw(15) << stock[i].sellingPrice << endl;
    }
}

int main()
{
```

```
    srand((unsigned int)time(0));

    InventoryItem stock[ITEM_COUNT];

    cout << "Enter details for " << ITEM_COUNT
         << " inventory items:\n";

    for (int i = 0; i < ITEM_COUNT; i++)
    {
        cout << "\nItem " << i + 1 << endl;
        stock[i] = insertItem();
    }

    displayStock(stock);

    return 0;
}
```

Task 01 – Explanation and sample

Key points:

• **struct InventoryItem** groups the four fields of each product.
• **insertItem()** reads the name and cost, generates the code, calculates the selling price, and returns the completed item.
• **substr(0, 3)** extracts the first three characters; **toupper()** converts them to uppercase.
• **to_string()** converts the running number to text so it can be joined to the prefix.
• **displayStock()** prints the 10 stored items in columns.

Example code format

```
Computer  -> COM_3121
Hard Disk -> HAR_3122
Keyboard  -> KEY_3123

Selling price = Cost + (Cost x 40%)
Selling price = Cost x 1.40
```

Task 02 – Carpet Cost Calculator (10 Marks)

This program asks for the number of rooms, creates a dynamic array of **RoomDimension** objects, reads the length and width of each room in feet and inches, and calculates carpet cost at **Rs. 18 per square foot**. The menu displays each room's cost or the total cost for the house.

Complete C++ code

```
#include <iostream>
#include <iomanip>
using namespace std;

const double PRICE_PER_SQ_FOOT = 18.0;

struct FeetInches
{
    int feet;
    double inches;
};

struct RoomDimension
{
    FeetInches length;
    FeetInches width;
};

// Converts a feet-and-inches measurement to decimal feet.
double toFeet(const FeetInches& measurement)
{
    return measurement.feet + measurement.inches / 12.0;
}

// Calculates carpet cost for one room.
double computeTheCarpetingCost(const RoomDimension& room)
{
    double length = toFeet(room.length);
    double width = toFeet(room.width);
    double area = length * width;

    return area * PRICE_PER_SQ_FOOT;
}

int main()
{
    int numberOfRooms;

    cout << "Enter number of rooms: ";
    cin >> numberOfRooms;

    if (numberOfRooms <= 0)
    {
        cout << "Invalid number of rooms!" << endl;
        return 0;
    }

    // Dynamic array: its size depends on the user's input.
    RoomDimension* rooms = new RoomDimension[numberOfRooms];

    for (int i = 0; i < numberOfRooms; i++)
    {
        cout << "\nRoom " << i + 1 << endl;
```

```
        cout << "Enter length (feet inches): ";
        cin >> rooms[i].length.feet >> rooms[i].length.inches;

        cout << "Enter width (feet inches): ";
        cin >> rooms[i].width.feet >> rooms[i].width.inches;
    }

    int choice;
    do
    {
        cout << "\n--------- CARPET COST MENU ---------\n";
        cout << "1. Display cost for each room\n";
        cout << "2. Display total cost for the house\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << fixed << setprecision(2);

            for (int i = 0; i < numberOfRooms; i++)
            {
                double cost = computeTheCarpetingCost(rooms[i]);
                cout << "Room " << i + 1
                     << " carpet cost = Rs. " << cost << endl;
            }
        }
        else if (choice == 2)
        {
            double totalCost = 0.0;

            for (int i = 0; i < numberOfRooms; i++)
            {
                totalCost += computeTheCarpetingCost(rooms[i]);
            }

            cout << fixed << setprecision(2);
            cout << "Total carpet cost = Rs. "
                 << totalCost << endl;
        }
        else if (choice != 3)
        {
            cout << "Invalid menu choice!" << endl;
        }

    } while (choice != 3);

    // Release the dynamic array.
    delete[] rooms;

    return 0;
}
```

Task 02 – Explanation and sample

Key points:

• **FeetInches** stores a measurement as feet and inches.
• **RoomDimension** contains two FeetInches members: length and width (nested structures).
• **new RoomDimension[numberOfRooms]** creates a dynamic array based on the user's room count.
• **toFeet()** converts inches into a fraction of a foot: feet + inches / 12.
• **computeTheCarpetingCost()** multiplies length by width and then by Rs. 18.
• **delete[] rooms** releases the dynamic array.

Sample calculation from the assignment

```
Length = 12 feet
Width  = 10 feet
Area   = 12 x 10 = 120 square feet
Cost   = 120 x Rs. 18 = Rs. 2,160
```

Note: The assignment image shows only page 1 of 2. The code above follows all requirements visible on that page, including the specified function prototype. If page 2 contains additional requirements or the exact provided structure definitions, those should be checked against this solution before final submission.